

UNIT-1: Introduction to SQLite

1.1 SQLite

- SQLite is a public domain software which allows RDBMS.
- **“SQLite is an in-process library that implements small, fast, self-contained, serverless, zero-configuration, transactional SQL database engine.”**
 - **Small:** It is small in size. (Lightweight)
 - **Fast:** Its process is fast as compare to other RDBMS.
 - **In process library:** It is a C-language library whose size is in KBs. It requires minimum support of C library functions like memset(), memcpy(), memcmp(), strcmp(), malloc(), free(), realloc().
 - **Self-contained:** It requires minimum support of external libraries. It requires minimum support of OS so it can run on any OS.
 - **Serverless:** It doesn't have any server process. It is not client server architecture. It has processes to read and write database directly from the database files on disk. The program which is able to access the disk is able to use SQLite.
 - **Zero-configuration:** Not a single configuration file requires. It doesn't need to install before use. There is no server process that needs to start, stop or configure.
 - **Transactional:** All transactions are fully Atomic, Consistent, Isolated and Durable (ACID). All changes within single transaction are either completed fully or not.

1.1.1 Difference between Oracle and SQLite

Oracle	SQLite
It is Structural Query language used to query a RDBMS written in C language.	It is embedded RDBMS written in ANSI-C.
It is server based.	It is file based.
It is client server architecture.	It is not client server architecture.
It is embedded in Server.	It is embedded in Client.
It is large in size.	It is small in size.
More configuration requires.	Doesn't require any configuration.
It supports stored procedures	It doesn't support stored procedures.
It is a language which is used with RDBMS like Oracle, MYSQL, MS Access etc. It is not a database.	It is portable database resource. It is a RDBMS which itself use SQL.

1.1.2 SQLite Advantages

The advantages of SQLite are listed below:

1. Lightweight

It is light weighted database so it is easy to use it as an embedded software with devices like televisions, Mobile phones, cameras.

2. Fast

Reading and writing data from database is fast as compare to other file processing systems. It simply loads the required data in place of loading the whole file. If the part of file is edited then only the respective parts of the file is modified.

3. No installation and configuration require

No installation as well as configuration require. There is a need to download the respective library file in computer to work with database.

4. Reliable

During power failure or system crash, there is no data loss because SQLite continuously stores data.

5. Portable

Because of minimum support from library, SQLite can work with any OS and with any type of applications. It can also be used with all programming language.

6. Accessible

This database can be accessible by any tools.

7. No server interference

It is not a client server-based architecture so there is not any kind of interference of server.

1.1.3 SQLite Features:

1. All transactions are fully Atomic, Consistent, Isolated and Durable (ACID). All changes within single transaction are either completed fully or not even after system crash or power failure.
2. No installation and configuration require.
3. All SQL statements are implemented in it with some additional features.
4. Its process is fast as compare to other file processing systems.
5. It is portable means it can work with all OS and with all programming languages.
6. It is serverless.
7. It is opensource software so it is freely available.
8. Complete database can be stored in a single cross-platform disk file.
9. It is easy to store data.
10. It is cross-platform DBMS. It doesn't require a large range of different platforms so it can be embedded within small devices like mobile, television.
11. The column length is variable means the size of the field's value is not fixed but it totally depends on value which you want to store.

1.1.4 SQLite Fundamentals:

1. Process to start SQLite:

- a. Open “command prompt”
 - b. Apply “cd..” command to move cursor to C:\ prompt.
 - c. Apply “cd sqlite” command to move cursor to SQLite folder.
 - d. Write “sqlite3” command to move to SQLite prompt (means we run the sqlite3.exe file).
2. Dot commands: (No need semicolon at the end)
- a. To exit from SQLite prompt: .quit or .exit
 - b. To see help: .help
 - c. To view the system settings: .show
 - d. To see the existing tables of database: .tables
 - e. To see the structure of table: .schema table_name
 - f. To display the database: .database
 - g. To display field heading during output: . header on/off
 - h. To open the database: .open database_name
 - i. To set output mode: .mode MODE
 - i. MODE can be: csv, column, html, insert etc.
- Sqlite3 student.db

1.1.5 SQLite Datatypes

- SQLite uses **dynamic datatype** system.
- The value which we store in column (field) determines column’s datatype.
- It is not compulsory to specify the datatype to the column at the time of table creation.
- Ex: If you specify the integer datatype with rollno column, then also text, real, null etc. values can also be allowed to store in rollno column.
- SQLite uses **manifest type** to store values of any type in a column.
- Manifest type means that a data type is a property of a value stored in a column, not the property of the column in which the value is stored.
- SQLite allows basic five datatypes which are referred as **Storage data class**.

1.1.5.1 SQLite storage data class:

1. NULL: It is used to specify the NULL value.
2. INTEGER: It is used to store signed integer. As per the value, the data is stored using 1,2,4,6 or 8 bytes.
3. REAL: It is used to store real values (floating point values). It uses 8 bytes to store it.
4. TEXT: It is used to store string using the database encoding (UTF-8, UTF-16BE or UTF-16LE- “Unicode Transformation Format”- It was designed to represent ASCII characters)
5. BLOB: It is used to store BLOB of data. (Large amount of text at one place). It stores exactly as per the input.

Note:

1. No Date & Time datatype available:

No specific datatype is available for date and time. It can be stored as TEXT, REAL or INTEGER. Date in TEXT stores in format of "yyyy-mm-ddhh:mm:ss"

2. No Boolean datatype available:

No specific datatype is available for Boolean. Simple INTEGER datatype with 0 (means false) or 1 (means true) value are used as Boolean datatype.

3. SQLite type interpretation:

Based on column value, the SQLite interpret the datatype as per following rules:

- Integer: If no quotes available, no decimal available.
- Text: If single or double quotes are available.
- Real: If no quotes available and decimal is available.
- Null: If NULL is written without quotes.
- BLOB: If X'ABCD' or x'abcd' is available.

4. typeof(value) function:

It is used to check the type of specified value.

Example:

```
create table temp (dt);
insert into temp values(1), (24), ('DP'), (24.5), ('RDBMS'), (x'abcd'), (NULL),
(X'ABCD'), ("BCA"),(15.6);
select dt, typeof(dt) from temp;
```

dt	typeof(dt)
-----	-----
1	integer
24	integer
DP	text
24.5	real
RDBMS	text
??	blob
	null
??	blob
BCA	text
15.6	real

5. Sorting data:

When the sorting is applied to this column, then sorting is applied as per given order NULL, Integer & Real, Text, BLOB.

Example:

```
select dt, typeof(dt) from temp order by dt;
```

dt	typeof(dt)
-----	-----
	null
1	integer
15.6	real
24	integer
24.5	real
BCA	text
DP	text
RDBMS	text
??	blob
??	blob

1.1.5.2 SQLite Affinity type:

- The preferred storage class for column is called its “affinity”.
- SQLite recommends the type to the column but it is not compulsorily required to specify the same type. It provides flexibility of data.
- Each column can assign any one of the following Affinity types:
 - TEXT: This affinity type can store text, null, blob data. If numerical data is inserted then first it is converted to text and then it is stored.
 - NUMERIC: All five storage classes data can be inserted under this affinity type.
 - INTEGER: It allows INT, INTEGER, TINYINT, SMALLINT, MEDIUMINT, BIGINT, UNSIGNED, BIG INT, INT2, INT8 type of data.
 - REAL: It allows all numeric values but it forces integer values to store as floating-point value.
 - NONE: If NONE is specified then no single data type is at first priority.

1.1.6 Transaction, Rollback, Commit

- Transaction is a set of operations perform as a unit. It can apply more than one changes to the database.
- Multiple SQLite queries are grouped together to perform specific task. Then it is considered as single transaction.
- SQLite is a transactional database where all changes are Atomic, Consistent, Isolated and Durable (ACID).
 - Atomic: It takes care about “Transaction should be fully completed or fully aborted.”
 - Consistent: It takes care about “The data within database remains consistent.”
 - Isolated: Each transaction must be separately executed without disturbing others. (Independent to each other)
 - Durable: If any kind of system failure occurs then also the system can give persistent result.
- SQLite transaction is either completed fully or not at all.
- Normally, SQLite is **auto-commit mode**. It starts processes and commits transaction automatically.
- Explicitly transactions can be controlled using following statements:
 1. BEGIN:

Using BEGIN statement, the transaction can start explicitly.
This transaction ends when ROLLBACK or COMMIT statement is explicitly found.
Syntax:

BEGIN; or BEGIN TRANSACTION;
 2. COMMIT:

To save the data permanently in database the COMMIT statement is written.
Syntax:

COMMIT; or COMMIT TRANSACTION;

3. ROLLBACK:

If we don't want to save data in database then ROLLBACK statement is written.

Syntax:

ROLLBACK; or ROLLBACK TRANSACTION;

Example:

```
create table stud(rno, name);
```

```
insert into stud values(1,'Het'), (2,'Mohit'), (3,'Manthan'),(4,'Maya'),(5,'Geeta');
```

rno	name
1	Het
2	Mohit
3	Manthan
4	Maya
5	Geeta

1. begin;

```
delete from stud where rno<3;
```

```
select * from stud;
```

rno	name
3	Manthan
4	Maya
5	Geeta

```
rollback;
```

```
select * from stud;
```

rno	name
1	Het
2	Mohit
3	Manthan
4	Maya
5	Geeta

2. begin;

```
delete from stud where rno<3;
```

```
commit;
```

```
select * from stud;
```

rno	name
3	Manthan
4	Maya
5	Geeta

1.2 Date and time (Not in syllabus)

- Date and time related data type is not available. So it is stored in form of text, integer or real.
- If the date and time is specified in integer and real format then it is difficult to understand.

- It is written in Text datatype with the format “yyyy-mm-dd hh:mm:ss.sss”.
 - Create table t (dt);
- Some of the date and time related functions:

No	Function																				
1	DATETIME('now'): To display current date and time as per UTC (Universal Time Coordinates). DATETIME('now','localtime'); To display date and local time. <pre>select datetime('now'), datetime('now','localtime') from t; datetime('now') datetime('now','localtime') ----- 2021-07-01 00:31:55 2021-07-01 06:01:55</pre>																				
2	DATE(value) To display the date value. Ex: select date(datetime('now')) from t; <pre>date(datetime('now')) ----- 2021-07-01</pre>																				
3	TIME(value) To display the time value. Ex: select time(datetime('now','localtime')) from t; <pre>time(datetime('now','localtime')) ----- 06:14:54</pre>																				
4	STRFTIME(format, time_string, modifiers...) To extract specific information from given date and time. Formats: <table border="1"> <thead> <tr> <th>Formats</th><th>Description</th></tr> </thead> <tbody> <tr> <td>%Y</td><td>Returns year value</td></tr> <tr> <td>%m</td><td>Returns month value</td></tr> <tr> <td>%d</td><td>Returns day value</td></tr> <tr> <td>%H</td><td>Returns hour value</td></tr> <tr> <td>%M</td><td>Returns minutes value</td></tr> <tr> <td>%S</td><td>Returns second value</td></tr> <tr> <td>%j</td><td>Returns day of year (1 to 366)</td></tr> <tr> <td>%w</td><td>Returns day of week in numbers (0 to 6) 0-Sunday</td></tr> <tr> <td>%W</td><td>Returns week of year (0 to 53)</td></tr> </tbody> </table> Examples: Select strftime('%Y', 'now'); Output: 2021 Select strftime('%m %d', 'now'); Output: 07 01 Select strftime('%w', '2021-05-12'); Output: 3	Formats	Description	%Y	Returns year value	%m	Returns month value	%d	Returns day value	%H	Returns hour value	%M	Returns minutes value	%S	Returns second value	%j	Returns day of year (1 to 366)	%w	Returns day of week in numbers (0 to 6) 0-Sunday	%W	Returns week of year (0 to 53)
Formats	Description																				
%Y	Returns year value																				
%m	Returns month value																				
%d	Returns day value																				
%H	Returns hour value																				
%M	Returns minutes value																				
%S	Returns second value																				
%j	Returns day of year (1 to 366)																				
%w	Returns day of week in numbers (0 to 6) 0-Sunday																				
%W	Returns week of year (0 to 53)																				
5	CURRENT_DATE To display the current date. Example: Select current_date; Output: 2021-07-01																				

1.3 Data Filtering

- Data filtering is applied to display or to fetch the specific information as per user's requirement.
- SELECT statement is used for data filtering.
- If user wants to display records as per his requirement, then the respective condition is given in SELECT statement.
- Example:

```
create table tblemp (Eid, Fname, Lname, Jdate, Salary, M_id, post);
```

```
insert into tblemp values
```

```
(101, 'Hemant', 'Sharma', '1995-01-21', 50000, 0, 'Manager'),
(102, 'Rita', 'Gandhi', '1997-12-20', 45000, 0, 'Manager'),
(103, 'Maya', 'Mistry', '2001-11-12', 30000, 101, 'Programmer'),
(104, 'Riya', 'Patel', '2003-09-15', 28000, 102, 'Programmer'),
(105, 'Shreya', 'Patel', '2010-10-17', 5000, 101, 'Peon'),
(106, 'Karan', 'Patel', '2015-08-18', 5000, 102, 'Peon'),
(107, 'Reyanshi', 'Vyas', '2011-06-05', 15000, 101, 'Clerk'),
(108, 'Mehul', 'Mehta', '2010-05-06', 17000, 102, 'Clerk'),
(109, 'Krupali', 'Patil', '2015-04-08', 10000, 101, 'Accountant'),
(110, 'Maitry', 'Vyas', '2017-03-11', 10000, 102, 'Accountant'),
(111, 'Mohan', 'Mehta', '2020-02-27', 15000, 101, 'Electrician');
```

Eid	Fname	Lname	Jdate	Salary	M_id	post
101	Hemant	Sharma	1995-01-21	50000	0	Manager
102	Rita	Gandhi	1997-12-20	45000	0	Manager
103	Maya	Mistry	2001-11-12	30000	101	Programmer
104	Riya	Patel	2003-09-15	28000	102	Programmer
105	Shreya	Patel	2010-10-17	5000	101	Peon
106	Karan	Patel	2015-08-18	5000	102	Peon
107	Reyanshi	Vyas	2011-06-05	15000	101	Clerk
108	Mehul	Mehta	2010-05-06	17000	102	Clerk
109	Krupali	Patil	2015-04-08	10000	101	Accountant
110	Maitry	Vyas	2017-03-11	10000	102	Accountant
111	Mohan	Mehta	2020-02-27	15000	101	Electrician

1.3.1 Filtering:

1.3.1.1 Where

- WHERE clause is used to write condition within query statement.
- Syntax:
SELECT * FROM table_name WHERE condition;
- Example:
select * from tblemp where M_id=101;

Eid	Fname	Lname	Jdate	Salary	M_id	post
103	Maya	Mistry	2001-11-12	30000	101	Programmer
105	Shreya	Patel	2010-10-17	5000	101	Peon
107	Reyanshi	Vyas	2011-06-05	15000	101	Clerk
109	Krupali	Patil	2015-04-08	10000	101	Accountant
111	Mohan	Mehta	2020-02-27	15000	101	Electrician

1.3.1.2 Distinct

- DISTINCT is used to display only the unique values or unique records of the specified table.
- Syntax:
SELECT DISTINCT columns FROM table_name WHERE condition;
- Example:
select distinct lname from tblemp;

```
Lname
-----
Sharma
Gandhi
Mistry
Patel
Vyas
Mehta
Patil
```

1.3.1.3 Between

- BETWEEN is a range searching operator. It is used to specify the range of values.
- Syntax:
SELECT columns FROM table_name WHERE col_name BETWEEN value1 AND value2;
- Example:
select * from tblemp where salary between 10000 and 20000;

Eid	Fname	Lname	Jdate	Salary	M_id	post
107	Reyanshi	Vyas	2011-06-05	15000	101	Clerk
108	Mehul	Mehta	2010-05-06	17000	102	Clerk
109	Krupali	Patil	2015-04-08	10000	101	Accountant
110	Maitry	Vyas	2017-03-11	10000	102	Accountant
111	Mohan	Mehta	2020-02-27	15000	101	Electrician

1.3.1.4 In

- IN operator is used to compare a column value with multiple values.
- Wild card characters are not allowed by IN operator.
- Syntax:
SELECT columns FROM table_name WHERE col_name IN (val1, val2,...);
- Example:
select * from tblemp where lname in ('Patel', 'Mehta', 'Vyas');

Eid	Fname	Lname	Jdate	Salary	M_id	post
104	Riya	Patel	2003-09-15	28000	102	Programmer
105	Shreya	Patel	2010-10-17	5000	101	Peon
106	Karan	Patel	2015-08-18	5000	102	Peon
107	Reyanshi	Vyas	2011-06-05	15000	101	Clerk
108	Mehul	Mehta	2010-05-06	17000	102	Clerk
110	Maitry	Vyas	2017-03-11	10000	102	Accountant
111	Mohan	Mehta	2020-02-27	15000	101	Electrician

1.3.1.5 Like

- Pattern matching operator (LIKE) is used to compare one string with another string.
- It allows matching of wildcard characters (%, _).
 - o ' % ': To match string with any numbers of characters including 0.
 - o ' _ ': To match any single character. If you want to match fixed length then multiple ' _ ' can be used Ex: ' M _ _ ' will match a string which is followed by any three characters.

- Syntax:

SELECT columns FROM table_name WHERE col_name LIKE compare_string;

- Example:

select * from tblemp where fname like 'M%';

Eid	Fname	Lname	Jdate	Salary	M_id	post
103	Maya	Mistry	2001-11-12	30000	101	Programmer
108	Mehul	Mehta	2010-05-06	17000	102	Clerk
110	Maitry	Vyas	2017-03-11	10000	102	Accountant
111	Mohan	Mehta	2020-02-27	15000	101	Electrician

select * from tblemp where fname like ' _ _ _ ';

Eid	Fname	Lname	Jdate	Salary	M_id	post
102	Rita	Gandhi	1997-12-20	45000	0	Manager
103	Maya	Mistry	2001-11-12	30000	101	Programmer
104	Riya	Patel	2003-09-15	28000	102	Programmer

1.3.1.6 Limit

- LIMIT clause is used to limit the total numbers of rows given by SELECT statement.
- It is optional part of SELECT statement.
- Normally, it is used with ORDER BY clause. This ORDER BY clause must be written before LIMIT clause in Query.

- Syntax:

SELECT * FROM table_name1 WHERE condition ORDER BY column
LIMIT row_count;

- Example:

select * from tblemp order by salary desc limit 3;

Eid	Fname	Lname	Jdate	Salary	M_id	post
101	Hemant	Sharma	1995-01-21	50000	0	Manager
102	Rita	Gandhi	1997-12-20	45000	0	Manager
103	Maya	Mistry	2001-11-12	30000	101	Programmer

- OFFSET keyword can also be used with LIMIT to start extracting rows from specific number. (Means from 11th to 20th).

- Syntax:

```
SELECT * FROM table_name1 WHERE condition ORDER BY column
LIMIT row_count OFFSET skip_rows;
```

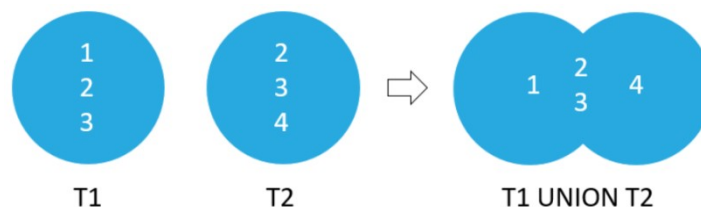
- Example:

```
select * from tblemp order by salary desc limit 5 offset 3;
```

Eid	Fname	Lname	Jdate	Salary	M_id	post
104	Riya	Patel	2003-09-15	28000	102	Programmer
108	Mehul	Mehta	2010-05-06	17000	102	Clerk
107	Reyanshi	Vyas	2011-06-05	15000	101	Clerk
111	Mohan	Mehta	2020-02-27	15000	101	Electrician
109	Krupali	Patil	2015-04-08	10000	101	Accountant

1.3.1.7 Union

- UNION operator is used to combine multiple rows of different tables by removing duplicate rows.
- When UNION is applied between two tables then numbers of columns with same type of data must be there.



- Total numbers of rows in each table can be of different length.

- Syntax:

```
SELECT * FROM table_name1 WHERE condition
UNION
SELECT * FROM table_name2 WHERE condition;
```

- Example:

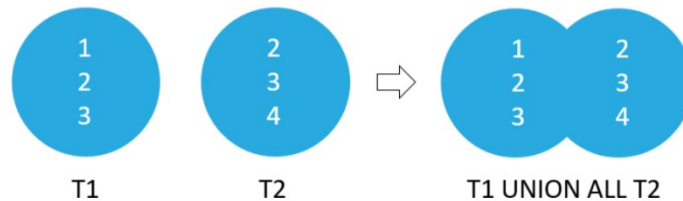
```
select eid, fname, salary from tblemp where salary < 20000;
select eid, fname, salary from tblemp where lname='Patel';
```

Eid	Fname	Salary	Eid	Fname	Salary
---	-----	-----	---	-----	-----
105	Shreya	5000	104	Riya	28000
106	Karan	5000	105	Shreya	5000
107	Reyanshi	15000	106	Karan	5000
108	Mehul	17000			
109	Krupali	10000			
110	Maitry	10000			
111	Mohan	15000			

```
select eid, fname, salary from tblemp where salary < 20000
UNION
select eid, fname, salary from tblemp where lname='Patel';
```

Eid	Fname	Salary
---	-----	-----
104	Riya	28000
105	Shreya	5000
106	Karan	5000
107	Reyanshi	15000
108	Mehul	17000
109	Krupali	10000
110	Maitry	10000
111	Mohan	15000

- UNION ALL is used to combine all rows of tables including duplicate rows.



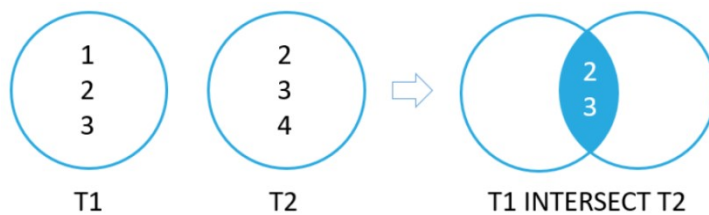
- Example:
- Different fields' data in one column: If same type of data is available in column, then these two columns values can be displayed in one column.
- Example:

```
select eid, fname, salary from tblemp where salary < 20000
union all
select eid, fname, salary from tblemp where lname='Patel';
```

Eid	Fname	Salary
---	-----	-----
105	Shreya	5000
106	Karan	5000
107	Reyanshi	15000
108	Mehul	17000
109	Krupali	10000
110	Maitry	10000
111	Mohan	15000
104	Riya	28000
105	Shreya	5000
106	Karan	5000

1.3.1.8 Intersect

- INTERSECT operator is used to compare two tables and display only the common rows.
- When INTERSECT is applied between two tables then numbers of columns with same type of data must be there.



- Syntax:

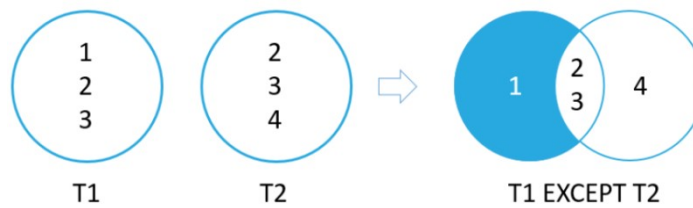
```
SELECT * FROM table_name1 WHERE condition
INTERSECT
SELECT * FROM table_name2 WHERE condition;
```
- Example:

```
select eid, fname, salary from tblemp where salary < 20000
intersect
select eid, fname, salary from tblemp where lname='Patel';
```

Eid	Fname	Salary
---	-----	-----
105	Shreya	5000
106	Karan	5000

1.3.1.9 Except

- EXCEPT operator is used to display the distinct records of First table which are not available in Second table.
- When EXCEPT is applied between two tables then numbers of columns with same type of data must be there.



- Syntax:
SELECT * FROM table_name1 WHERE condition
EXCEPT
SELECT * FROM table_name2 WHERE condition;
- Example:
select eid, fname, salary from tblemp where salary < 20000
except
select eid, fname, salary from tblemp where lname='Patel';

Eid	Fname	Salary
---	-----	-----
107	Reyanshi	15000
108	Mehul	17000
109	Krupali	10000
110	Maitry	10000
111	Mohan	15000

1.3.1.10 IS NULL

- IS NULL operator is used to check whether given column value is NULL or not.
- It returns 1 if the value is NULL else it returns 0.
- Syntax:
SELECT * FROM table_name2 WHERE col_name IS NULL;
- Example:
- IS NOT NULL operator returns 1 if value is not NULL and returns 0 if value is NULL.
- Example:
create table tblperson(pid,pname);
insert into tblperson values (1,'Raj'),(2,NULL), (3,'Preet'),(4,NULL),(5,'Ronit');

pid	pname
---	-----
1	Raj
2	
3	Preet
4	
5	Ronit

```
select * from tblperson where pname is null;
```

pid	pname
---	-----
2	
4	

```
select * from tblperson where pname is not null;
```

pid	pname
---	-----
1	Raj
3	Preet
5	Ronit

1.3.2 Group by

- GROUP BY clause is used to group the records based on specified column.
- AGGREGATE functions can be applied with GROUP BY clause.
- It allows user to perform task on group of records.
- Syntax:

```
SELECT column, aggregate_function(column) FROM tablename
WHERE condition
GROUP BY column
```

- Example:

```
select m_id, count(m_id) as total_count from tblemp group by m_id;
```

M_id	total_count
----	-----
0	2
101	5
102	4

1.3.3 Having

- HAVING clause is used to check the condition like WHERE.
- It uses only with GROUP BY clause.
- The condition which is specified with HAVING clause must use the AGGREGATE function.
- Syntax:

```
SELECT column, aggregate_function(column) FROM tablename
WHERE condition
GROUP BY column
HAVING aggregate_function (column) operator value
```

- Example:

```
select m_id, count(m_id) as total_count from tblemp group by m_id having
count(m_id)>3;
```

M_id	total_count
----	-----
101	5
102	4

1.3.4 Order by

- ORDER BY clause is used to sort multiple records either in ascending order or in descending order.
- By default, it sorts the records in ascending order. If you want to sort the records in descending order then DESC keyword is used with ORDER BY clause.

- Syntax:
SELECT columns FROM tablename WHERE condition ORDER BY column [DESC];
- Example:
select * from tblemp order by lname, fname;

Eid	Fname	Lname	Jdate	Salary	M_id	post
---	-----	-----	-----	-----	----	-----
102	Rita	Gandhi	1997-12-20	45000	0	Manager
108	Mehul	Mehta	2010-05-06	17000	102	Clerk
111	Mohan	Mehta	2020-02-27	15000	101	Electrician
103	Maya	Mistry	2001-11-12	30000	101	Programmer
106	Karan	Patel	2015-08-18	5000	102	Peon
104	Riya	Patel	2003-09-15	28000	102	Programmer
105	Shreya	Patel	2010-10-17	5000	101	Peon
109	Krupali	Patil	2015-04-08	10000	101	Accountant
101	Hemant	Sharma	1995-01-21	50000	0	Manager
110	Maitry	Vyas	2017-03-11	10000	102	Accountant
107	Reyanshi	Vyas	2011-06-05	15000	101	Clerk

1.3.5 Conditional Logic (CASE)

- CASE expression is used to display results as per conditions.
- It performs like IF-THEN-ELSE statement.
- It can be used with WHERE, HAVING, ORDER BY, SELECT, UPDATE, DELETE.
- Syntax:
CASE
 WHEN condition1 THEN output1
 WHEN condition2 THEN output2
 ...
 [ELSE output_n]
END;
- Here, first of all condition1 is evaluated if it is true then output1 is selected else condition2 is evaluated and so on. When none of the conditions are true then it selects the output_n. If ELSE is not specified then CASE returns NULL.
- Example:
select eid, fname, salary, (case when salary < 10000 then 1000 when salary < 20000 then 2000 when salary < 30000 then 3000 else 5000 end) as Bonus from tblemp;

Eid	Fname	Salary	Bonus
---	-----	-----	-----
101	Hemant	50000	5000
102	Rita	45000	5000
103	Maya	30000	5000
104	Riya	28000	3000
105	Shreya	5000	1000
106	Karan	5000	1000
107	Reyanshi	15000	2000
108	Mehul	17000	2000
109	Krupali	10000	2000
110	Maitry	10000	2000
111	Mohan	15000	2000

1.4 SQLite joins:

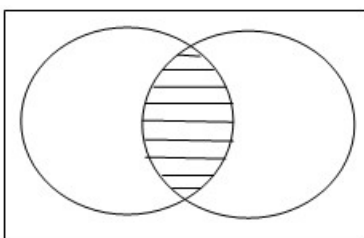
- SQLite joins are used to retrieve data from multiple tables.
- Tables:

STUD_INFO			
Rno	Name	Marks	Did
1	Mayank	35	1
2	Maya	25	2
3	Mohit	45	1
4	Raghav	10	10
5	Ankit	49	3

DEPT	
Did	Dname
1	IT
2	Management
3	Science
4	Commerce
5	Arts

1.4.1 Inner Join

- When the tables are combined using Inner Join then it returns all those records which are matched with both tables.



- USING(column) can be used in place of ON for comparing same column name.
- Example:

ANSI Style (Only two tables can join)

```
SELECT NAME, DNAME FROM DEPT
INNER JOIN STUD_INFO
ON DEPT.DID=STUD_INFO.DID;
```

```
SELECT name, dname FROM dept
INNER JOIN stud_info
```

USING(did);

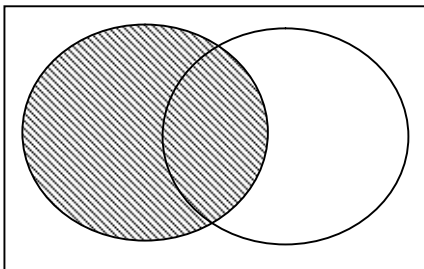
Theta style: (Can Join multiple tables)

```
SELECT NAME, DNAME FROM DEPT, STUD_INFO
WHERE DEPT.DID=STUD_INFO.DID;
```

NAME	DNAME
Mayank	IT
Maya	Management
Mohit	IT
Ankit	Science

1.4.2 Left Join

- When the tables are combined using Left Join then all the records of Left tables are displayed.
- If the related record's information is not available within another table then it returns NULL to those fields



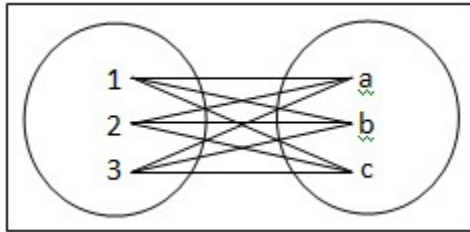
- Example:

```
SELECT NAME, DNAME FROM DEPT
LEFT JOIN STUD_INFO
ON DEPT.DID=STUD_INFO.DID;
```

NAME	DNAME
Mayank	IT
Maya	Management
Mohit	IT
Ankit	Science
	Arts
	Commerce

1.4.3 Cross Join

- When tables are combined with Cross Join then it displays all records.
- Each record of first table is joined with all records of another table.
- If first table contains N records and second table contains M records then Cross Join table displays $N \times M$ records.



- Example:

```
SELECT NAME, DNAME FROM STUD_INFO
CROSS JOIN DEPT;
```

NAME	DNAME	NAME	DNAME	NAME	DNAME
Mayank	IT	Mohit	Management		
Mayank	Management	Mohit	Science		
Mayank	Science	Mohit	Commerce		
Mayank	Commerce	Mohit	Arts		
Mayank	Arts	Raghav	IT		
Maya	IT	Raghav	Management		
Maya	Management	Raghav	Science		
Maya	Science	Raghav	Commerce		
Maya	Commerce	Raghav	Arts		
Maya	Arts	Ankit	IT		
Mohit	IT	Ankit	Management		
		Ankit	Science		
		Ankit	Commerce		
		Ankit	Arts		

1.4.4 Self Join

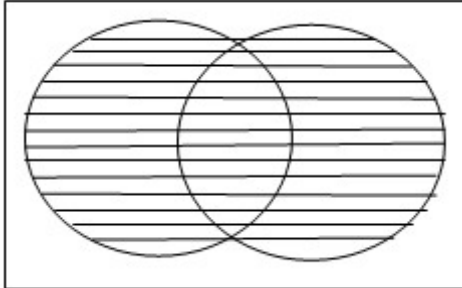
- Self Join combines the table with itself.
- Same table is renamed as twice and then these two tables are combined with each other.
- Example:

```
SELECT A.RNO AS ARNO, B.NAME AS BNAME, A.MARKS AS AMARKS FROM
STUD_INFO A, STUD_INFO B WHERE A.MARKS < B.MARKS;
```

ARNO	BNAME	AMARKS
1	Mohit	35
1	Ankit	35
2	Mayank	25
2	Mohit	25
2	Ankit	25
3	Ankit	45
4	Mayank	10
4	Maya	10
4	Mohit	10
4	Ankit	10

1.4.5 Full outer joins

- When tables are combined using Full Join then it displays all the records of both the tables and returns NULL to the fields where the respective information is missing.
- It gives combine result of LEFT and RIGHT Joins.



- Example:

```
SELECT NAME, DNAME FROM DEPT
FULL JOIN STUD_INFO
ON DEPT.DID=STUD_INFO.DID;
```

NAME	DNAME
Mayank	IT
Maya	Management
Mohit	IT
Ankit	Science
	Arts
	Commerce
Raghav	

1.5 SQLite Trigger:

- Trigger is an object which automatically executes when INSERT, UPDATE or DELETE operation is performed on Database.
- It requires for data validation before inserting or updating.
- FOR EACH ROW:
Means trigger is applied for each row. Example: if in table, trigger is applied with 5 rows then trigger is applied 5 times.
- FOR EACH STATEMENT:
Means trigger is applied once per table. Example: if in table, trigger is applied with 5 rows then trigger is applied 1 time.
- SQLite trigger is called **FOR EACH ROW** trigger.
- When the table is dropped then automatically, the respective triggers are dropped.

1.5.1 CREATE trigger:

- Syntax:
CREATE TRIGGER trigger_name
[BEFORE|AFTER] [INSERT|UPDATE|DELETE]

```

ON table_name
[WHEN condition]
BEGIN
    Statements;
END;

```

- BEFORE & AFTER keywords are used to specify when you want to execute Trigger.
 - BEFORE keyword is used to fire trigger before operation execution on database.
 - AFTER keyword is used to fire trigger after operation execution on database.
- INSERT, UPDATE and DELETE are the three operations where the trigger can fire automatically.
- WHEN keyword is used to specify the condition. If the condition becomes true then only the trigger is executed. If it is not mentioned then trigger is executed for each row.
- BEGIN & END are used to specify the trigger logic (What you want to perform automatically)
- OLD and NEW are the two references. They are used to access the data of row. Data is called using NEW.col_name and OLD.col_name.
 - NEW: It is used with INSERT and UPDATE operations.
 - OLD: It is used with UPDATE and DELETE operations.
- RAISE() function may be used to generate user defined error.
- **Example: The manager_id must be either 101 or 102. If any other value is inserted then need to display error message.**

```

CREATE TRIGGER trg_mid
BEFORE INSERT ON tblemp
BEGIN
    SELECT
    CASE
        WHEN NEW.m_id NOT IN (101,102) THEN
            RAISE(ABORT, 'Invalid Manager ID')
    END;
END;

```

Check:-----

```

insert into tblemp values(112, 'Raj', 'Sharma', '21-02-1996', 50000, 103, 'Peon');
(Generate user defined error)

```

```
insert into tblempvalues(112, 'Raj', 'Sharma', '21-02-1996', 50000, 102, 'Peon');
(Accepted)
```

1.5.2 Drop trigger

- To drop the trigger, DROP statement is used.
- Syntax:
DROP TRIGGER trigger_name;
- Example:
DROP TRIGGER trg_mid;

1.5.3 Listing all triggers

- To see the list of all triggers created in database, sqlite_master table is used.
- Syntax:
SELECT name FROM sqlite_master WHERE type='trigger';
- If you want to see the list of all triggers created for a table then you can specify the multiple conditions.
SELECT name FROM sqlite_master WHERE type='trigger' and tbl_name='.....';
(write name of table)

1.5.4 Disable and Enable trigger

- Enable and disable trigger is not supported by SQLite.
- DISABLE is used to disable the specific trigger.
- Syntax:
ALTER TRIGGER trigger_name DISABLE;
- ENABLE is used to enable the specific trigger if it is disable by user.
- Syntax:
ALTER TRIGGER trigger_name ENABLE;

1.6 SQLite Constraint (Not in syllabus)

1.6.1 Primary key:

- This constraint is used to create primary key.
- Example:

```
create table dept(did number primary key, dname text);

insert into dept values(1,'Marketing');
insert into dept values(1,'Marketing'); error
```

1.6.2 Foreign key:

- This constraint is used to create foreign key (Means to establish relation between parent and child table).
- Before creating table there is a need to write following statement in SQLite.
PRAGMA foreign_keys=ON;

- Example:

```
PRAGMA foreign_keys = ON;
create table dept(did number primary key, dname text);
insert into dept values(1, 'Marketing'),(2,'Finance'), (3,'HR'),(4,'IT');
create table emp(eid number, ename text, did number references dept(did));
insert into emp values(101,'Janvi',1);
insert into emp values(102,'Jensi',5); error

delete from dept where did=1; error
```

- To allow user to delete record from parent table as well as related records from child table, there is a need to set ON DELETE CASCADE with create table.
- Example:

```
create table emp(eid number, ename text, did number references dept(did) on
delete cascade);
insert into emp values(101,'Janvi',1);

delete from dept where did=1; (delete data from both tables)
```

1.6.3 NOT NULL:

- To restrict user to compulsory fill value in particular column, NOT NULL constraint is used.
- Example:

```
create table emp (eno number not null, ename text);
insert into emp values(1,'Preet');
insert into emp(ename) values('Reet'); error
```

1.6.4 UNIQUE:

- To insert the unique values in particular column, UNIQUE constraint is used.
- Example:

```
create table emp (eno number unique, ename text);
insert into emp values(1,'Reet');
insert into emp values(1,'Preet'); error
```

- Example:

```
create table emp (eno number, ename text, phno text, unique(eno, phno));
insert into emp values(1,'Preet','2323434334');
insert into emp values(1,'Preet','2323434334'); error
insert into emp values(2,'Preet','2323434334'); No error
```


1.6.5 DEFAULT:

- To set the default value when user didn't enter any value in column, then DEFAULT constraint is used.
- Example:

```
create table emp (eno number not null, ename text, city text default 'Surat');
insert into emp(eno,ename) values(1,'Reet');
select * from emp;
```

1.6.6 CHECK:

- To check the condition before storing data permanently within table, CHECK constraint is used.
- Example: (Allow to insert salary greater than 10000 and the city must be one of the "Surat", "Anand" and "Navsari".)

```
create table emp (eno number unique, ename text, salary number
check(salary>10000), city text check(city in ('Surat','Anand','Navsari')));
insert into emp values(1,'Reet', 15000,'Surat');
insert into emp values(2,'Reet', 5000,'Surat'); error
insert into emp values(3,'Reet', 15000,'Aurat'); error
```

- Example: (Check the length of mobile number)

```
create table emp (eno number unique, ename text, phno text
check(length(phno)=10));
insert into emp values(1,'Reet', '1234567891');
insert into emp values(2,'Reet', '1234'); error
```